

Vidya Pratishthan's Kamalnayan Bajaj Institute of Engineering

& Technology, Baramati

Department of Artificial Intelligence and Data Science

Lab Manual for Operating System

1.	Write a series of BASH shell commands for common system administration tasks, including file management, process control, disk usage monitoring, and system information retrieval.
2.	Implement a program using fork(), exec(), and wait() system calls to create a child process ,execute a new program in the child, and synchronize the parent process to wait for the child to complete.
3.	Implement preemptive (Round Robin) and non-preemptive (FCFS, SJF, Priority) CPU scheduling algorithms, calculating and displaying waiting time, turnaround time, and CPU utilization for each.
4.	Implement a simulation of virtual memory using demand paging, and apply page replacement algorithms such as FIFO (First-In-First-Out),LRU (Least Recently Used) and optimal to handle page faults and minimize them.
5.	Implement various disk scheduling algorithms, including FIFO (First-Come-First-Serve),SSTF (Shortest Seek Time First), SCAN, and C-SCAN, and evaluate their performance based on average seek time.
6.	Build a basic file system that supports file creation, reading, writing, and deletion using low-level system calls such as open(), read(), write(), and close().
7.	Implement the Banker's algorithm to avoid deadlock by analysing resource allocation and safely granting resource requests.
8.	Write a program using semaphores to implement the Readers-Writers problem, ensuring mutual exclusion while allowing multiple readers and one writer to access a shared resource concurrently.

9.	Write a C program to demonstrate mutual exclusion using mutex locks, solving the critical section problem.
10.	Write a C program to dynamically adjust the priority of a process using the nice() system call and observe the impact on scheduling.

Assignment No.1

Title: Write a series of BASH shell commands for common system administration tasks, including file management, process control, disk usage monitoring, and system information retrieval.

Aim:

To understand and explore common BASH shell commands used for system administration tasks, including file management, process control, disk usage monitoring, system information retrieval, user management, network management, security administration, log management, package management, and automation with scripting.

Theory:

The **BASH (Bourne Again SHell)** is a command-line interpreter widely used in Linux and UNIX-based systems for system administration tasks. It provides powerful tools for managing files, monitoring processes, handling users, securing the system, and automating repetitive tasks. Below is an in-depth discussion of various system administration tasks and their related BASH commands.

1. File Management

File management involves creating, modifying, organizing, and securing files and directories. A system administrator often needs to manage a large number of files efficiently.

Command	Description
ls	Lists files and directories.
cp	Copies files and directories.
mv	Moves or renames files and directories.
rm	Deletes files and directories.
mkdir	Creates a new directory.
rmdir	Removes an empty directory.
touch	Creates an empty file or updates its timestamp.
find	Searches for files based on name, size, or modification time.
tar	Archives multiple files into a single file.
zip/unzip	Compresses or extracts files using ZIP format.

2. Process Control

Process management ensures efficient resource allocation and system performance. System administrators must monitor, start, stop, or adjust process priorities.

Command	Description
ps	Displays running processes.
top	Shows real-time system resource usage.
htop	A user-friendly alternative to top.
kill	Terminates a process using its PID.
pkill	Kills a process by name.
nohup	Runs a command in the background and prevents termination upon logout.
nice	Starts a process with a given priority.
renice	Changes the priority of a running process.

3. Disk Usage Monitoring

Managing disk space is crucial to prevent system slowdowns and failures. These commands help monitor and clean up storage.

Command	Description
df -h	Displays free and used disk space in human-readable format.
du -sh	Shows the size of a directory or file.
lsblk	Lists all storage devices and partitions.
mount	Mounts a filesystem.
umount	Unmounts a filesystem.
fsck	Checks and repairs filesystem errors.
blkid	Retrieves information about disk partitions.

4. System Information Retrieval

System administrators need to gather details about system status, kernel version, uptime, and memory usage.

Command	Description
uname -a	Displays system information, including kernel version.
uptime	Shows how long the system has been running.
free -m	Displays memory usage in megabytes.
whoami	Prints the current user.
who	Lists all logged-in users.
hostname	Displays the system's hostname.
lscpu	Shows CPU details.
dmidecode	Provides detailed hardware information.

5. User Management

User management ensures proper access control and permission settings.

Command	Description
adduser	Creates a new user.
deluser	Deletes a user.
passwd	Changes a user's password.
usermod	Modifies user attributes.
groups	Shows the groups a user belongs to.
id	Displays user and group ID.
chown	Changes the ownership of a file or directory.
chmod	Changes file permissions.

Conclusion:

BASH shell commands are essential for system administrators to efficiently manage files, processes, disk space, users, networks, security, logs, and software packages. Mastering these commands improves productivity, ensures security, and enhances system stability in Linux and UNIX environments.

Assignment No.2

Title: Implement a program using fork(), exec(), and wait() system calls to create a child process, execute a new program in the child, and synchronize the parent process to wait for the child to complete.

Aim:

To implement a program in C that demonstrates process creation using fork(), executing a new program in the child process using exec(), and synchronizing the parent process using wait().

Theory:

Process Management in UNIX/Linux:

A **process** is an executing instance of a program. Every running application in an operating system is a process, and each process has a unique identifier called a **Process ID (PID)**.

There are two main types of processes:

- **Parent Process:** The process that creates another process.
- **Child Process:** A new process created by the parent using the fork() system call.

In UNIX/Linux, process management includes three fundamental system calls:

1. **fork()** – Used to create a new process.
2. **exec()** – Used by a process to replace its memory space with a new program.
3. **wait()** – Used by the parent to pause execution until the child process completes.

2. fork() System Call – Process Creation:

The fork() system call is responsible for creating a new process. It creates an exact copy of the parent process with a separate memory space. The execution of both the parent and child process starts immediately after fork() is called.

Behavior of fork():

- The parent process gets the PID of the child process as the return value.
- The child process gets 0 as the return value.
- If process creation fails, fork() returns -1.

Algorithm for fork():

1. Start the parent process.
2. Call fork() to create a child process.
3. If fork() succeeds:
    - The child process runs separately.
    - The parent process continues execution.
4. If fork() fails, print an error message.
5. End the process.

3. exec() System Call – Executing a New Program:

The exec() system call replaces the currently running process image with a new program. This is useful when the child process needs to execute a different program than the parent.

Variants of exec()

Function	Description
execl()	Takes a path and a list of arguments.
execv()	Takes a path and an array of arguments.
execlp()	Searches for the executable in PATH.
execvp()	Similar to execv() but searches in PATH.

Algorithm for exec():

1. Start a process (typically a child process after fork()).
2. Call an exec() function to execute a new program.
3. If exec() succeeds, the process image is replaced with the new program.
4. If exec() fails, the process continues execution from the next statement after exec().
5. End the process.

4. wait() System Call – Synchronization Between Parent and Child:

The wait() system call is used by a parent process to wait for the completion of its child process. It ensures that the parent does not continue execution until the child process terminates.

Algorithm for wait():

1. Start the parent process.
2. Call fork() to create a child process.
3. The parent process calls wait().
4. The child process executes a task (e.g., using exec()).
5. Once the child process finishes execution, the parent resumes.
6. End the process.

5. Complete Algorithm Using fork(), exec(), and wait():

1. Start the parent process.
2. Call fork() to create a child process.
3. If fork() is successful:
    - The child process:
        - Calls exec() to execute a new program.
        - If exec() fails, print an error message.
    - The parent process:
        - Calls wait() to wait for the child process to complete.
4. Once the child process finishes execution, the parent resumes execution.
5. End the process.

Practical Applications:

- **Multitasking:** Running multiple processes simultaneously.
- **Process Isolation:** Ensuring different programs do not interfere with each other.
- **Shell Commands Execution:** Running commands in the terminal using fork() and exec().
- **Web Servers:** Handling multiple client requests by forking child processes.

Conclusion:

The fork(), exec(), and wait() system calls are essential for process management in UNIX/Linux operating systems. fork() allows process creation, exec() enables program execution, and wait() ensures synchronization. Understanding these concepts is crucial for efficient multitasking and system programming.

Assignment No.3

Title: Implement preemptive (Round Robin) and non-preemptive (FCFS, SJF, Priority) CPU scheduling algorithms, calculating and displaying waiting time, turnaround time, and CPU utilization for each.

Aim:

To implement and analyze different CPU scheduling algorithms, including both preemptive (Round Robin) and non-preemptive (FCFS, SJF, Priority) scheduling, by calculating waiting time, turnaround time, and CPU utilization.

Theory:

CPU Scheduling in Operating Systems

CPU scheduling is the process of selecting a process from the ready queue for execution by the CPU. It is essential for efficient multitasking and resource utilization in an operating system.

CPU scheduling algorithms can be categorized into:

- **Non-Preemptive Scheduling:** Once a process is assigned the CPU, it runs to completion.
- **Preemptive Scheduling:** The CPU can be taken away from a running process based on scheduling criteria

A). FIRST COME FIRST SERVE:

To calculate the average waiting time using the FCFS algorithm first the waiting time of the first process is kept zero and the waiting time of the second process is the burst time of the first process and the waiting time of the third process is the sum of the burst times of the first and the second process and so on. After calculating all the waiting times the average waiting time is calculated as the average of all the waiting times. FCFS mainly says first come first serve the algorithm which came first will be served first.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process name and the burst time

Step 4: Set the waiting of the first process as 0 and its burst time as its turnaround time .

Step 5: for each process in the Ready Q calculate

a). Waiting time (n) = waiting time (n-1) + Burst time (n-1)

b). Turnaround time (n)= waiting time(n)+Burst time(n)

Step 6: Calculate

a) Average waiting time = Total waiting Time / Number of process

b) Average Turnaround time = Total Turnaround Time / Number of process

Step 7: Stop the process.

B). SHORTEST JOB FIRST: (Non- Preemption)

To calculate the average waiting time in the shortest job first algorithm the sorting of the process based on their burst time in ascending order then calculate the waiting time of each process as the sum of the bursting times of all the process previous or before to that process.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Start the Ready Q according the shortest Burst time by sorting according to lowest to highest burst time.

Step 5: Set the waiting time of the first process as 0 and its turnaround time as its burst time.

Step 6: Sort the processes names based on their Burt time

Step 7: For each process in the ready queue, calculate

a) Waiting time(n)= waiting time (n-1) + Burst time (n-1)

b) Turnaround time (n)= waiting time(n)+Burst time(n)

Step 8: Calculate

c) Average waiting time = Total waiting Time / Number of process

d)Average Turnaround time = Total Turnaround Time / Number of process

Step 9: Stop the process.

C). PRIORITY:

To calculate the average waiting time in the priority algorithm, sort the burst times according to their priorities and then calculate the average waiting time of the processes. The waiting time of each process is obtained by summing up the burst times of all the previous processes.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Sort the ready queue according to the priority number.

Step 5: Set the waiting of the first process as 0 and its burst time as its turnaround time

Step 6: Arrange the processes based on process priority

Step 7: For each process in the Ready Q calculate

Step 8: for each process in the Ready Q calculate

a) Waiting time(n)= waiting time (n-1) + Burst time (n-1)

b) Turnaround time (n)= waiting time(n)+Burst time(n)

Step 9: Calculate

c) Average waiting time = Total waiting Time / Number of process

d) Average Turnaround time = Total Turnaround Time / Number of process

Print the results in an order.

Step 10: Stop

D)ROUND ROBIN:

To aim is to calculate the average waiting time. There will be a time slice, each process should be executed within that time-slice and if not it will go to the waiting state so first check whether the burst time is less than the time-slice. If it is less than it assign the waiting time to the sum of the total times. If it is greater than the burst-time then subtract the time slot from the actual burst time and increment it by time-slot and the loop continues until all the processes are completed.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue and time quantum (or) time slice

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Calculate the no. of time slices for each process where No. of time slice for process (n) = burst time process (n)/time slice

Step 5: If the burst time is less than the time slice then the no. of time slices =1.

Step 6: Consider the ready queue is a circular Q, calculate

a) Waiting time for process (n) = waiting time of process(n-1)+ burst time of process(n-1) + the time difference in getting the CPU from process(n-1)

b) Turnaround time for process(n) = waiting time of process(n) + burst time of process(n)+ the time difference in getting CPU from process(n).

Step 7: Calculate

c) Average waiting time = Total waiting Time / Number of process

d) Average Turnaround time = Total Turnaround Time / Number of process

Step 8: Stop the process

Conclusion:

The choice of scheduling algorithm affects system performance:

- **FCFS** is simple but causes the **convoy effect**.
- **SJF** minimizes waiting time but can lead to **starvation**.
- **Priority Scheduling** ensures important processes execute first but may cause **starvation**.
- **Round Robin** provides fairness but may result in **higher average waiting times**.

By implementing these algorithms, we can analyze their efficiency in different scenarios and choose the most suitable one based on system requirements.

Assignment No.4

Title: Implement a simulation of virtual memory using demand paging, and apply page replacement algorithms such as FIFO (First-In-First-Out),LRU (Least Recently Used) and optimal to handle page faults and minimize them.

Aim:

To implement a simulation of virtual memory using demand paging and apply page replacement algorithms such as FIFO (First-In-First-Out), LRU (Least Recently Used), and Optimal to handle page faults and minimize them.

Theory:

1. Virtual Memory and Demand Paging

Virtual memory is a memory management technique that allows a process to execute even if it is not fully loaded into the main memory (RAM). It extends the available memory by using secondary storage (disk) as additional memory space.

Demand Paging is a technique where pages are loaded into memory only when required. If a page is needed but is not in memory, a page fault occurs, and the operating system must bring the required page from disk into memory.

Steps in Demand Paging:

1. A process requests a page.
2. If the page is in memory, the request is granted immediately.
3. If the page is not in memory:
    - A page fault occurs.
    - The operating system selects a free frame or uses a page replacement algorithm if no free frame is available.
    - The required page is brought from secondary storage into memory.
    - The page table is updated.

2. Page Replacement Algorithms

When a page fault occurs and there are no free frames, the operating system uses a page replacement algorithm to decide which page to remove.

a) First-In-First-Out (FIFO)

- The oldest page in memory is removed first.
- Simple but inefficient, as it does not consider the frequency of use.

Algorithm for FIFO Page Replacement:

1. Start.
2. Maintain a queue of pages in memory.
3. When a page fault occurs:
    - If there is space, insert the new page.
    - If full, remove the oldest page (first in queue) and insert the new page.
4. Update the page table.
5. Repeat for all page requests.
6. Display the number of page faults.
7. End.

b) Least Recently Used (LRU)

- The page that has not been used for the longest time is replaced.
- More efficient than FIFO, but requires tracking usage history.

Algorithm for LRU Page Replacement:

1. Start.
2. Maintain a record of page usage timestamps.
3. When a page fault occurs:
    - If there is space, insert the new page.
    - If full, remove the least recently used page.
4. Update the page table.
5. Repeat for all page requests.
6. Display the number of page faults.
7. End.

c) Optimal Page Replacement

- Replaces the page that will not be used for the longest time in the future.
- Gives the best performance, but requires future knowledge, which is not possible in real systems.

Algorithm for Optimal Page Replacement:

1. Start.
2. When a page fault occurs:
    - If there is space, insert the new page.
    - If full, check all pages in memory and predict future usage.
    - Replace the page that will not be used for the longest time in the future.
3. Update the page table.
4. Repeat for all page requests.
5. Display the number of page faults.
6. End.

3. Performance Metrics

Each algorithm is evaluated based on:

Metric	Formula
Page Fault Rate	(Total Page Faults / Total Page Requests) * 100%
Hit Ratio	(Total Page Hits / Total Page Requests) * 100%

Conclusion:

- FIFO is simple but may remove important pages.
- LRU is better as it considers past usage but requires additional tracking.
- Optimal provides the least page faults but is not practical in real systems.

By simulating these algorithms, we can analyze their efficiency in different scenarios and optimize memory management for better performance.

Assignment No.5

Title: Implement various disk scheduling algorithms, including FIFO (First-Come-First-Serve), SSTF (Shortest Seek Time First), SCAN, and C-SCAN, and evaluate their performance based on average seek time.

Aim:

To implement and analyze different disk scheduling algorithms, including **FIFO (First-Come-First-Serve), SSTF (Shortest Seek Time First), SCAN, and C-SCAN**, by calculating the average seek time for each.

Theory:

1. Disk Scheduling in Operating Systems

Disk scheduling is the process of deciding the order in which disk I/O requests are processed to optimize performance. Since disk access is **much slower** than CPU and memory operations, efficient disk scheduling reduces **seek time**, improving overall system performance.

Key Terms:

- **Seek Time:** The time required for the disk arm to move to the requested track.
- **Total Seek Time:** The sum of all individual seek times for a given set of requests.
- **Average Seek Time:** Average Seek Time= Total Seek Time /Number of Requests

2. Disk Scheduling Algorithms

Different algorithms decide the sequence in which disk requests are processed.

a) First-Come-First-Serve (FCFS)

- Requests are processed in the order they arrive.
- **Simple but inefficient** if requests are far apart, leading to **long seek times**.

Algorithm for FCFS Disk Scheduling:

1. Start.
2. Accept disk requests in the order they arrive.
3. Process each request sequentially.

4. Calculate the **seek time** as the absolute difference between consecutive requests.
5. Compute **total and average seek time**.
6. Display the results.
7. End.

b) Shortest Seek Time First (SSTF)

- The request closest to the current disk head position is served next.
- **Reduces seek time but may cause starvation** for far requests.

Algorithm for SSTF Disk Scheduling:

1. Start.
2. Sort disk requests based on the shortest seek time from the current head position.
3. Select the closest request and process it.
4. Repeat until all requests are processed.
5. Compute **total and average seek time**.
6. Display the results.
7. End.

c) SCAN (Elevator Algorithm)

- The disk arm moves **in one direction**, serving requests until the end of the disk.
- Then, it reverses direction and processes remaining requests.
- **Prevents starvation and provides fair scheduling.**

Algorithm for SCAN Disk Scheduling:

1. Start.
2. Arrange requests in ascending order.
3. Move the disk arm **in one direction** and serve requests.
4. When the edge of the disk is reached, reverse direction and continue processing.
5. Compute **total and average seek time**.
6. Display the results.

7. End.

d) C-SCAN (Circular SCAN)

- Similar to SCAN, but **only moves in one direction**.
- After reaching the end, it **jumps back to the beginning** without serving any requests.
- **Provides uniform wait times**.

Algorithm for C-SCAN Disk Scheduling:

1. Start.
2. Arrange requests in ascending order.
3. Move the disk arm **in one direction**, serving requests.
4. When the edge of the disk is reached, **jump back to the start**.
5. Continue serving requests in the same direction.
6. Compute **total and average seek time**.
7. Display the results.
8. End.

3. Performance Evaluation

Each algorithm is evaluated based on:

Metric	**Formula**
Total Seek Time	Sum of all seek times
Average Seek Time	(Total Seek Time) / (Number of Requests)
Fairness	Uniform servicing of requests without starvation

Conclusion:

In this way,by analyzing these algorithms, we can determine the best approach based on **seek time and fairness**, improving disk performance in real-world applications.

Assignment No.6

Title: Build a basic file system that supports file creation, reading, writing, and deletion using low-level system calls such as open(), read(), write(), and close().

Aim:

To develop a basic file system that supports **file creation, reading, writing, and deletion** using low-level system calls such as open(), read(), write(), and close().

Theory:

1. Introduction to File Systems

A **file system** is responsible for managing how data is stored and retrieved on a storage device. It organizes data into files and directories, providing access control and efficient data management.

A file system typically supports basic operations such as:

- **Creating files**
- **Reading files**
- **Writing to files**
- **Deleting files**

2. Low-Level File System Calls in UNIX/Linux

Low-level file system calls provide direct interaction with the **operating system's kernel** for file manipulation. Unlike high-level file operations (e.g., using C++/Python file handling), low-level system calls offer **fine-grained control** over file operations.

System Call	Description
open()	Opens an existing file or creates a new one.
read()	Reads data from a file into memory.
write()	Writes data from memory into a file.
close()	Closes an open file to free system resources.
unlink()	Deletes a file from the file system.

3. File Operations Using System Calls

a) File Creation (open())

- The open() system call is used to create a new file if it doesn't exist.
- It requires **file name, access mode, and permissions**.
- Example permissions:
  - 0644 → Owner can read/write, others can read.
  - 0777 → Everyone has full access.

Working of open()

1. The function returns a **file descriptor** (a unique number assigned to the opened file).
2. If open() fails, it returns -1, indicating an error.

b) Writing to a File (write())

- The write() system call writes data to a file.
- It requires a **file descriptor, buffer (data to write), and size**.

Working of write()

1. The function writes data from the buffer to the file.
2. It returns the **number of bytes written**.
3. If writing fails, it returns -1.

c) Reading from a File (read())

- The read() system call reads data from a file into a buffer.
- It requires a **file descriptor, buffer, and size**.

Working of read()

1. Reads data into a buffer.
2. Returns the **number of bytes read**.
3. If **end-of-file (EOF)** is reached, it returns 0.

d) Closing a File (close())

- The close() system call **releases the file descriptor**, freeing system resources.

Working of close()

1. Ensures that all file operations are saved before closing.
2. Returns 0 on success or -1 on failure.

e) Deleting a File (unlink())

- The unlink() system call removes a file from the file system.

Working of unlink()

1. Deletes a file **permanently**.
2. If the file is **open**, it will be deleted after closing.

Conclusion:

By using **low-level system calls**, we can implement a basic file system that supports **file creation, reading, writing, and deletion**. These system calls interact directly with the **operating system's kernel**, providing better control over file operations compared to high-level APIs. Understanding these concepts is essential for **systems programming, operating system development, and file system design**.

Assignment No.7

Title: Implement the Banker's algorithm to avoid deadlock by analysing resource allocation and safely granting resource requests.

Aim:

To implement the **Banker's Algorithm** to prevent deadlocks by analyzing resource allocation and safely granting resource requests in a multi-process system.

Theory:

1. Introduction to Deadlock

A **deadlock** occurs when multiple processes hold resources and wait indefinitely for additional resources held by other processes, causing the system to freeze. To prevent deadlocks, the **Banker's Algorithm** ensures that resource allocation is done **safely** without leading to an unsafe state.

2. What is the Banker's Algorithm?

The **Banker's Algorithm**, developed by **Edsger Dijkstra**, is a **deadlock avoidance** algorithm that ensures a system never enters an unsafe state. It works similarly to a bank providing loans while ensuring it can always satisfy some requests.

Key Characteristics:

- Used in **operating systems** to prevent deadlocks.
- Works by simulating resource allocation before actually granting resources.
- If granting resources leads to an **unsafe state**, the request is **denied**.

3. Components of the Banker's Algorithm

The algorithm uses the following data structures:

Term	Description
Available	An array storing the number of available instances for each resource.
Max	A matrix storing the **maximum demand** of each process for each resource.
Allocation	A matrix storing the **currently allocated** resources to each process.
Need	A matrix storing the remaining resource requirements of each process.

The **Need** matrix is calculated as: $Need[i][j] = Max[i][j] - Allocation[i][j]$

4. Safety Algorithm (Checking for a Safe State)

This algorithm determines whether the system is in a **safe state** by checking if all processes can complete execution.

Algorithm:

1. **Initialize:**
    - Create a Finish[] array for all processes and set all values to **false** (indicating no process has completed execution).
    - Copy the Available[] array into a new Work[] array.
    - Initialize an empty SafeSequence[].
2. **Find a Process:**
    - Search for a process P[i] such that:
        - Need[i] \leq Work (The process's remaining resource needs can be met).
        - Finish[i] == false (The process has not completed).
3. **If such a process exists:**
    - Allocate its resources back: Work = Work + Allocation[i].
    - Mark Finish[i] = true.
    - Append P[i] to SafeSequence[].
4. **Repeat Steps 2-3** until:
    - Either all processes are finished (Finish[i] == true for all i), meaning the system is in a **safe state**, or
    - No such process can be found, meaning the system is in an **unsafe state**.
5. **If all processes are finished:**
    - Print the **safe sequence**.
    - Otherwise, declare that the system is in an **unsafe state**.

5. Resource-Request Algorithm (Granting Resource Requests Safely)

This algorithm determines whether a resource request by a process can be safely granted.

Algorithm:

1. **Check Validity of Request:**

- If Request[i] > Need[i]:
  - **Error:** The process is requesting more than its declared maximum.
  - **Terminate.**

2. **Check Availability of Resources:**
   - If Request[i] > Available:
     - **Process must wait** (resources are currently unavailable).
     - **Terminate.**
3. **Pretend to Allocate Resources:**
   - Available = Available - Request[i]
   - Allocation[i] = Allocation[i] + Request[i]
   - Need[i] = Need[i] - Request[i]
4. **Perform Safety Check:**
   - Run the **Safety Algorithm** to check if the system is still in a **safe state** after allocation.
5. **Decision:**
   - If the system remains in a **safe state**, grant the request permanently.
   - If the system enters an **unsafe state**, rollback the allocation:
     - Available = Available + Request[i]
     - Allocation[i] = Allocation[i] - Request[i]
     - s
     - **Deny the request.**

Conclusion:

- The **Banker's Algorithm** ensures that the system never enters a **deadlock state** by checking **resource allocation safety** before granting requests.
- It helps maintain **system stability** and ensures **fair resource distribution** among processes.
- However, it requires prior knowledge of **maximum demands**, which may not always be practical in real-world systems.

Assignment No.8

Title: Write a program using semaphores to implement the Readers-Writers problem, ensuring mutual exclusion while allowing multiple readers and one writer to access a shared resource concurrently.

Aim:

To implement a solution for the **Readers-Writers Problem** using **semaphores**, ensuring **mutual exclusion** while allowing multiple readers and a single writer to access a shared resource concurrently.

Theory:

1. Introduction to the Readers-Writers Problem

The **Readers-Writers Problem** is a classic synchronization problem in operating systems that deals with resource sharing between multiple processes:

- **Readers** can access the resource **simultaneously** (multiple readers allowed).
- **Writers** require **exclusive access** (only one writer allowed at a time).
- **Conflict** arises when a writer and one or more readers try to access the shared resource at the same time.

This problem has two primary variations:

1. **First Readers-Writers Problem (No Starvation for Readers)**: A reader is never kept waiting if the resource is free.
2. **Second Readers-Writers Problem (No Starvation for Writers)**: Once a writer is ready, it must be given priority over new readers.

2. Solution Using Semaphores

Semaphores are used to ensure proper synchronization between **readers and writers**.

Semaphores Used:

- **mutex**: Ensures mutual exclusion for updating the **reader count**.
- **writeLock**: Ensures **only one writer** can write at a time.
- **readerCount**: Keeps track of the number of active readers.

Synchronization Rules:

1. **Reader Entry:**
    - Lock mutex to update readerCount.
    - If it's the first reader, block writers by locking writeLock.
    - Unlock mutex.
2. **Reader Exit:**
    - Lock mutex to update readerCount.
    - If it's the last reader, release writeLock (allowing writers to proceed).
    - Unlock mutex.
3. **Writer Entry:**
    - Lock writeLock (ensuring exclusive access to the resource).
4. **Writer Exit:**
    - Unlock writeLock (allowing other readers or writers to proceed).

Algorithm

Reader Process Algorithm (Pseudocode)

```
procedure Reader()
   wait(mutex)       // Lock mutex to update readerCount
   readerCount++
   if readerCount == 1 then
      wait(writeLock)  // First reader blocks writers
   signal(mutex)     // Release mutex

   // Critical Section: Reading the shared resource
   Read()

   wait(mutex)       // Lock mutex to update readerCount
   readerCount--
   if readerCount == 0 then
```

```
        signal(writeLock)  // Last reader allows writers
    signal(mutex)     // Release mutex
end procedure
```

Writer Process Algorithm (Pseudocode)

```
procedure Writer()
    wait(writeLock)                 // Lock writeLock to get exclusive access
    // Critical Section: Writing to the shared resource
    Write()
  signal(writeLock) // Release writeLock
end procedure
```

Conclusion:

- The **Readers-Writers Problem** is efficiently solved using **semaphores**, ensuring **mutual exclusion** and **concurrent reading**.
- **Multiple readers** can read the resource simultaneously, while **only one writer** can write at a time.
- This implementation avoids **race conditions** and ensures **data consistency** in concurrent systems.

Assignment No.9

Title: Write a C program to demonstrate mutual exclusion using mutex locks, solving the critical section problem..

Aim:

To write a C program that demonstrates mutual exclusion using **mutex locks**, solving the **critical section problem** by ensuring synchronized access to shared resources.

Theory:

1. Critical Section Problem

In **concurrent programming**, multiple processes or threads often need access to a **shared resource** (e.g., a file, database, or memory variable). If multiple threads access the resource simultaneously, it can lead to:

- **Race conditions** (unpredictable outcomes)
- **Data inconsistency**
- **Deadlocks** or **starvation**

To prevent these issues, mutual exclusion must be enforced using synchronization mechanisms like **mutex locks**.

2. Mutex Locks

A **mutex (mutual exclusion) lock** ensures that only one thread at a time can access the critical section.

Working Principle of Mutex

1. **Locking**: A thread must acquire the mutex lock before entering the critical section.
2. **Critical Section Execution**: The thread safely modifies the shared resource.
3. **Unlocking**: The thread releases the mutex lock after finishing.
4. **Other Threads Wait**: If another thread tries to access the critical section, it must wait until the mutex is released.

Mutex Functions in C (pthread library)

- pthread_mutex_init(&mutex, NULL); → Initializes the mutex.
- pthread_mutex_lock(&mutex); → Locks the mutex.
- pthread_mutex_unlock(&mutex); → Unlocks the mutex.
- pthread_mutex_destroy(&mutex); → Destroys the mutex when done.

3. Advantages of Mutex Locks

- Prevents race conditions and data corruption.
- Ensures **mutual exclusion** among competing threads.
- Simple to implement and efficient for protecting small critical sections.

4. Limitations

- If not used carefully, mutex locks can lead to **deadlocks** (when two or more threads wait indefinitely for each other to release locks).
- Can cause **priority inversion**, where a high-priority thread waits for a low-priority thread holding the mutex.

Algorithm

1. Initialize Mutex

1. Declare a pthread_mutex_t variable.
2. Initialize the mutex using pthread_mutex_init().

2. Create Multiple Threads

1. Use pthread_create() to spawn multiple threads.
2. Each thread should attempt to enter the critical section.

3. Implement Critical Section

1. Each thread:
    - Calls pthread_mutex_lock() to acquire the mutex.
    - Executes the **critical section** (modifies shared resource).
    - Calls pthread_mutex_unlock() to release the mutex.

4. Synchronization and Cleanup

1. The main thread waits for all worker threads using pthread_join().
2. Destroy the mutex using pthread_mutex_destroy().

Conclusion:

Mutex locks effectively enforce **mutual exclusion**, preventing race conditions. Proper use of mutex locks ensures that data remains consistent and processes execute safely in a concurrent environment.

Assignment No.10

Title: Write a C program to dynamically adjust the priority of a process using the nice() system call and observe the impact on scheduling.

Aim:

To write a C program that dynamically adjusts the priority of a process using the nice() system call and observes its impact on CPU scheduling.

Theory:

1. Process Scheduling and Priority

In a **multiprogramming** operating system, processes compete for CPU time. The scheduler decides which process gets CPU resources based on scheduling policies and **priority levels**.

- **Higher priority** processes get more CPU time.
- **Lower priority** processes get less CPU time.

The **priority** of a process can be modified dynamically using the nice() system call in Linux.

2. The nice() System Call

The nice() system call allows a process to adjust its **niceness value**, which affects its scheduling priority.

Syntax:

int nice(int increment);

- The increment parameter adjusts the process's current niceness value.
- A **higher niceness value** lowers the priority (process runs slower).
- A **lower niceness value** increases the priority (process runs faster).
- Default niceness value is **0**, ranging from **-20** (highest priority) to **19** (lowest priority).

3. Impact of nice() on Scheduling

- **Higher priority processes (-20)** execute faster and get more CPU time.
- **Lower priority processes (19)** execute slower and get less CPU time.
- The change in priority is observed by monitoring execution time using commands like top or ps.

4. Role of Priority in Process Management

Process priority plays a critical role in system performance and responsiveness. Some key factors include:

- **Real-Time Scheduling:** Critical tasks (e.g., multimedia processing, system services) require higher priority.
- **Background Processing:** Low-priority processes (e.g., backups, indexing) should not interfere with user experience.
- **Fair CPU Distribution:** The system balances CPU allocation between high-priority and low-priority tasks.
- **Preemptive Scheduling:** High-priority processes can preempt lower-priority ones for efficient CPU utilization.

5. Practical Use Cases of nice()

- Running computationally heavy tasks at low priority to avoid system lag.
- Increasing priority of real-time applications like video streaming and gaming.
- Scheduling batch jobs effectively to optimize CPU resource distribution.
- Managing server workloads by adjusting process priorities dynamically.

Algorithm

1. Parent Process

1. **Create a child process** using fork().
2. If fork() fails, print an error and terminate.
3. If fork() succeeds:
    - **Child Process:**
        1. Call nice(increment) to change the priority.
        2. Print the new priority.
        3. Execute a CPU-intensive task to observe scheduling effects.
    - **Parent Process:**
        1. Wait for the child process using wait().
        2. Print the completion message.

2. Child Process Execution

1. Modify its scheduling priority using nice().

2. Perform a CPU-intensive task (e.g., a loop-based computation).
3. Print the process completion message.

Conclusion:

- The nice() system call effectively changes a process's scheduling priority.
- Increasing niceness decreases CPU allocation, making the process run **slower**.
- Decreasing niceness increases CPU allocation, making the process run **faster**.
- This method is useful for controlling CPU-bound tasks in multi-user or real-time environments.
- Understanding and managing process priority can significantly enhance system efficiency and resource utilization.